

Proyecto Final: Inteligencia Artificial

Diseño y Evolución de un Agente Aspirador Autónomo: Desde la Inferencia Lógica Proposicional hasta la Gestión Probabilística de Riesgos

Facultad de Ingeniería

Universidad Nacional Autónoma de México (UNAM)

26 de mayo de 2026

Resumen

Este informe presenta el desarrollo, la arquitectura y la evolución tecnológica de un agente aspirador autónomo inteligente para la navegación y limpieza eficientes en entornos bidimensionales parcialmente observables. Se detalla el marco metodológico basado en la formalización de agentes inteligentes y el modelo PEAS. Asimismo, se describe la transición evolutiva del robot a través de tres fases distintas: desde un modelo lógico puro limitado a obstáculos estáticos, pasando por un Agente Basado en Conocimiento con detección inferencial de amenazas ocultas mediante deducción proposicional en SymPy, hasta culminar en un Agente Probabilístico robusto capaz de cuantificar la incertidumbre.

1. Introducción e Importancia de la Aspiración Autónoma

La higiene de los espacios habitables constituye un pilar fundamental para el bienestar del ser humano. La acumulación de polvo y macropartículas en superficies sólidas no solo degrada la estética ambiental, sino que impacta de manera directa en el desempeño cognitivo, los niveles de estrés emocional y la salud respiratoria a largo plazo.

Tradicionalmente, las tareas de aseo y mantenimiento de suelos se han gestionado de forma manual, lo que representa un consumo considerable de tiempo y energía humana en actividades altamente repetitivas. La introducción de la robótica de servicio y los agentes de limpieza autónomos surge como una solución tecnológica esencial para transferir esta carga operativa al hardware automatizado.

Sin embargo, las soluciones comerciales de bajo costo suelen apoyarse en arquitecturas de control puramente reactivas. Estos robots carecen de un modelo interno del mundo, operando mediante trayectorias pseudo-aleatorias y rebotando al colisionar físicamente con los obstáculos. Este comportamiento reactivo genera:

- Un gasto energético y una baja eficiencia en la cobertura de área por unidad de tiempo.
- Un desgaste acelerado de los componentes de hardware debido al impacto físico recurrente.
- Incapacidad para adaptarse lógicamente a entornos complejos y cambiantes.

Por el contrario, el diseño de un agente racional autónomo maximiza la utilidad del sistema a través de un mapeo ordenado, la planificación de trayectorias seguras y la conservación inteligente de recursos (batería), demostrando el valor práctico de los paradigmas avanzados de la Inteligencia Artificial.

2. Definición de Agente e Implementación de Modelos

Para estructurar de manera formal el comportamiento de nuestra aspiradora inteligente, adoptamos la definición de **Agente Racional** propuesta por Russell & Norvig. Un agente es cualquier entidad capaz de percibir su entorno a través de sensores y actuar sobre él mediante actuadores, orientando sus acciones hacia la maximización de una medida de rendimiento dada su secuencia de percepciones.

La formalización de este agente se define mediante la estructura **PEAS** (Performance, Environment, Actuators, Sensors):

2.1. Modelos de Inteligencia Artificial Integrados

Para lograr un comportamiento racional integral, la arquitectura del agente incorpora cuatro modelos computacionales especializados que interactúan de forma sinérgica:

1. **Base de Conocimientos (KB):** Un motor de inferencia lógica proposicional implementado en `logic.py`. Utiliza la librería `SymPy` para verificar la consecuencia

Cuadro 1: Especificación PEAS del Agente Aspirador

Componente	Descripción
Rendimiento (<i>Performance</i>)	Número de casillas de suciedad limpiadas, minimización de pasos totales ejecutados (eficiencia energética), evitación absoluta de colisiones con muros y minimización de ingresos a alfombras de alto riesgo.
Entorno (<i>Environment</i>)	Matriz discreta de tamaño $N \times M$ con obstáculos rígidos (muros), una estación base de recarga segura, áreas distribuidas con polvo (suciedad dinámica) y zonas con alfombras peligrosas.
Actuadores (<i>Actuators</i>)	Motor de traslación hacia adelante (MOVER), rotación sobre su eje (ROTAR_IZQ, ROTAR_DER), activador de succión (ASPIRAR) y comando de acoplamiento de carga (TERMINAR, RECARGAR).
Sensores (<i>Sensors</i>)	Sensor de proximidad frontal (Obstaculo_Frente), detector de suelo sucio (Suciedad), detector físico de impacto (Colision) y detector táctil de polvo fino perimetral (Pelusa).

lógica de hipótesis de seguridad a través de pruebas de refutación (verificación de insatisfacibilidad mediante satisfiable($KB \wedge \neg\alpha$) = False).

2. **Búsqueda en Grafos (BFS):** Algoritmo en `navigation.py` que encuentra la ruta óptima en celdas conectadas de forma ortogonal, traduciendo coordenadas cartesianas abstractas en comandos físicos del robot (giros y avances).
3. **Optimización Evolutiva (Algoritmos Genéticos):** Módulo en `genetic.py` diseñado para resolver el Problema del Agente Viajero (TSP) sobre las zonas con suciedad acumulada. Mediante operadores de selección por torneo, cruzamiento de orden (*Order Crossover*) y mutación por intercambio (*Swap Mutation*), planifica una ruta óptima de patrullaje.
4. **Clasificación Adaptativa (Machine Learning):** Árbol de decisión CART implementado en `ml_model.py`. Entrenado sobre 3,000 muestras sintéticas de supervivencia, clasifica dinámicamente el estado óptimo de comportamiento del agente (EXPLORE, GO_CHARGE, PATROL, GO_FINISH) basándose en la distancia a la base y la energía disponible.

3. Evolución Tecnológica del Agente

La evolución del sistema de navegación inteligente y toma de decisiones se estructuró en tres fases de desarrollo incremental, permitiendo aislar la complejidad computacional en cada iteración.

3.1. Fase 1: Mapeo Lógico con Muros Estáticos (Sin Zonas de Riesgo)

En la primera etapa, el entorno de simulación solo contenía muros rígidos estáticos distribuidos al azar. El agente operaba bajo una Base de Conocimientos simplificada, donde:

- Cada coordenada (x, y) descubierta se clasificaba de forma binaria como transitable o intransitable.
- Si el sensor frontal detectaba un obstáculo, el literal de ocupación $O_{x,y}$ se integraba directamente a la base de conocimientos como una proposición verdadera.
- El espacio no explorado (niebla de guerra) se consideraba seguro por defecto mientras no se detectara activamente un obstáculo en la dirección de la cámara de navegación.

Esta arquitectura básica garantizaba una tasa de colisiones nula frente a obstáculos detectados, pero era insuficiente para manejar amenazas ambientales no colisionales u ocultas.

3.2. Fase 2: Agente Basado en Conocimiento con Alfombras (Obstáculos Ocultos)

La segunda fase introdujo 'Alfombras' distribuidas en el mapa. Estas zonas representan áreas prohibidas y frágiles que el agente no debe de tocar. La característica crítica de estas alfombras es que **no son detectables mediante el sensor de colisión frontal**, por lo que el agente debe deducir su existencia de manera indirecta.

Para emular este problema, se diseñó una regla análoga a la brisa del Mundo del Wumpus: **la presencia de una alfombra genera una dispersión perimetral de polvo fino ("pelusa") en las celdas adyacentes**.

El agente aplica inferencia lógica proposicional estricta. Al percibir pelusa (P), construye axiomas de equivalencia lógica sobre el mapa en coordenadas relativas (ax, ay) :

$$P_{ax,ay} \iff (C_{ax+1,ay} \vee C_{ax-1,ay} \vee C_{ax,ay+1} \vee C_{ax,ay-1}) \quad (1)$$

Donde C representa la presencia confirmada de una alfombra.

Adicionalmente, se integra el **Axioma de Separación de Alfombras**, el cual establece que dos alfombras no pueden colocarse de forma contigua (ni siquiera en diagonal):

$$\forall(ax_1, ay_1), (ax_2, ay_2) \in \text{adyacentes}, \quad \neg(C_{ax_1,ay_1} \wedge C_{ax_2,ay_2}) \quad (2)$$

Antes de desplazarse hacia una celda frontera (fx, fy) , el agente formula una consulta lógica a la KB: **¿Se demuestra que no hay una alfombra en esta celda?** ($KB \models \neg C_{fx,fy}$). Si la deducción por refutación es exitosa, la celda se clasifica como totalmente segura.

3.3. Fase 3: Agente Probabilístico y de Gestión de Riesgo (VacuumAgentProb)

Aunque la inferencia deductiva formal es matemáticamente rigurosa, sufre de limitaciones bajo condiciones complejas o información incompleta. Si un agente deductivo

se encuentra rodeado de fronteras no demostrables como seguras ($\neg C_{fx,fy}$ no deducible debido a la falta de observaciones periféricas), el sistema experimenta un **bloqueo por indecisión (deadlock)**, deteniendo su operación de forma prematura.

Para solucionar esto, se desarrolló el **Agente Probabilístico** en `agentProb.py`. En esta fase, el agente abandona la lógica booleana bidireccional y modela el entorno como un campo de probabilidad de riesgos en cada celda $P(C_{x,y})$:

1. **Axioma de Supervivencia:** Si el agente está ubicado físicamente sobre una coordenada (x, y) y sigue operativo, se le asigna una probabilidad absoluta de riesgo de cero ($P(C_{x,y}) = 0,0$).
2. **Propagación Dinámica de Probabilidad:** Cuando el sensor detecta una pelusa en $P_{px,py}$, el agente distribuye uniformemente la sospecha entre las celdas vecinas que no han sido visitadas, no son muros y no se encuentran en la zona segura perimetral de una alfombra confirmada:

$$P(C_{nx,ny}) = P(C_{px,py}) + \frac{1,0}{|\text{Vecinos Válidos}|} \quad (3)$$

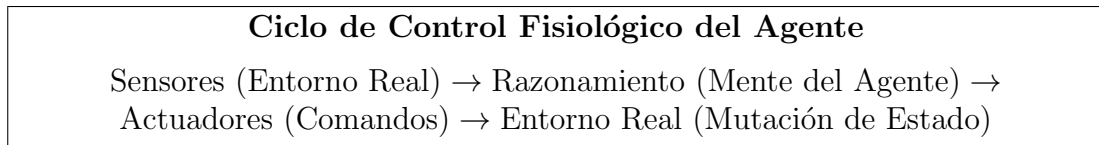
3. Clasificación de Amenazas por Umbral:

- $P(C_{x,y}) = 0,0 \rightarrow$ Celda totalmente segura.
- $0,0 < P(C_{x,y}) < 0,9 \rightarrow$ Celda bajo sospecha o riesgosa.
- $P(C_{x,y}) \geq 0,9 \rightarrow$ Alfombra confirmada (intransitable).

Este enfoque probabilístico permite al agente ordenar y jerarquizar sus planes de navegación. Si se agotan las celdas de seguridad absoluta, el agente evalúa los riesgos relativos y avanza con precaución sobre la trayectoria de menor probabilidad de peligro, evitando parálisis y garantizando la compleción de la tarea de exploración.

4. Flujo Continuo de la Simulación

El flujo operativo que gobierna la simulación sigue un lazo de control cerrado clásico de los sistemas inteligentes, ejecutado de manera secuencial y aislada en cada ciclo de la simulación principal (`main.py`):



El proceso detallado de cada fase del ciclo comprende:

4.1. Fase de Sensores (Percepción)

El entorno real (`environment.py`) evalúa la física del tablero y proporciona un conjunto de percepciones locales al agente a través de la interfaz:

```
1 percepts = env.get_percepts(agent.x, agent.y, agent.orientacion, env.
  agent_real_x, env.agent_real_y)
```

El agente recibe información binaria puntual sobre la casilla delantera, colisiones inmediatas, presencia de basura o presencia de pelusa perimetral, sin conocer en ningún momento la matriz absoluta del mapa.

4.2. Fase de Razonamiento (Actualización Mental y Planificación)

Una vez recibidas las percepciones:

1. **Actualización del Mapa Mental:** El agente mapea los datos sensoriales a sus estructuras de coordenadas relativas, registrando muros confirmados en `self.walls` y actualizando su distribución de riesgo de alfombras en `self.carpet_prob`.
2. **Evaluación del Estado por Machine Learning:** Invoca al clasificador del árbol de decisión para reevaluar la macro-estrategia según los pasos consumidos y la distancia actual a la estación de recarga:

```
1 self.mode = predict_mode(self.model, self.battery/self.  
    max_battery, dist_to_base, exp_done, pat_done)  
2
```

3. **Planificación BFS / Genética:**

- Si el modo es **EXPLORE**, busca celdas fronteras seguras no visitadas usando el camino BFS más corto.
- Si el modo es **PATROL**, ejecuta el algoritmo genético para calcular el ciclo Hamiltoniano de patrullaje óptimo sobre las basuras remanentes.
- Si el modo es **GO_CHARGE**, planifica la ruta directa de escape hacia la coordenada base (0,0).

4.3. Fase de Actuadores (Emisión de Comandos y Acción)

El agente extrae la primera acción atómica de su plan de navegación (secuencia en `self.path`). Esta acción física se devuelve al bucle de ejecución:

```
1 action = agent.think_and_act()
```

El entorno interpreta el comando y muta el estado real del mundo (desplaza el robot físicamente, elimina la suciedad del mapa si la acción es **ASPIRAR**, o recarga la batería si se encuentra acoplado en la base).

El visualizador de PyGame (`visualizer.py`) captura la mutación del entorno real en el panel izquierdo y dibuja la interpretación mental y los campos de riesgo calculados por el agente en el panel derecho, reiniciando inmediatamente el ciclo continuo.

5. Conclusión

El desarrollo de este agente inteligente demuestra el valor de integrar diferentes metodologías de la Inteligencia Artificial para resolver problemas prácticos complejos. A través del uso ordenado del modelo PEAS, se logró estructurar un robot eficiente y altamente adaptable. La transición evolutiva desde una lógica proposicional formal en SymPy hacia

un modelo de gestión probabilística de incertidumbre resolvió los problemas clásicos de deadlock por indecisión lógica. Finalmente, la sinergia de la búsqueda estructurada (BFS), la optimización bio-inspirada (Algoritmos Genéticos) y la toma de decisiones por aprendizaje automático (Árboles de Decisión) dotan al robot de un comportamiento racional y adaptativo que optimiza tanto el aseo del entorno como la conservación de recursos físicos esenciales.